第24课 阶段二复习与测试

一、课程信息

项目	内容
课次	第 24 课 / 共 40 课
主题	阶段二复习 + 综合测试
教学目标	① 系统梳理第 13-23 课全部知识点，帮助学生建立基础算法知识框架 ② 通过快速回顾典型例子唤醒记忆，查漏补缺 ③ 通过综合测试检验阶段二学习效果，为阶段三做准备
前置知识	第 13-23 课（函数、二维数组、枚举法、模拟法、冒泡排序、选择/插入排序、sort 与 STL、贪心算法、高精度加法）
后续衔接	第 25 课 字符串（一）char 数组与 string（进入阶段三）

二、课堂时间分配（共 100 分钟）

阶段	时间	内容
开场导入	5 分钟	说明今天的安排，鼓励学生信心
专题复习	40 分钟	七大专题系统梳理（每专题约 5-6 分钟）
答疑互动	5 分钟	自由提问、重点强调
综合测试	45 分钟	独立完成测试卷
收卷 + 预告	5 分钟	收卷、点评测试感受、预告下节课

三、开场导入（5 分钟）

教师话术

> 同学们，我们从第 13 课到第 23 课，已经学完了 C++ 的"基础算法阶段"！
>
> 回想一下，我们学了函数、二维数组、枚举法、模拟法、三种排序、sort 函数、贪心算法、高精度加法……内容可真不少！
>
> 今天这节课分两部分：
> - **前半节**：老师带大家把这些知识点像"整理工具箱"一样，分门别类理一遍。
> - **后半节**：做一份测试卷，检验一下大家的掌握情况。
>
> 测试不用紧张！错了正好知道哪里需要加强。考完之后，我们下节课就要进入阶段三——学"字符串"了！

四、七大专题复习（40 分钟）

知识地图总览

先在黑板/PPT 上画出整体框架，让学生看到全貌：

```
阶段二：基础算法（第13-23课）
│
├── 代码组织
│ └── 专题一：函数（定义、调用、参数、返回值）
│
├── 数据结构
│ └── 专题二：二维数组
│
├── 基础算法思想
│ ├── 专题三：枚举法
│ ├── 专题四：模拟法
│ └── 专题六：贪心算法
│
├── 排序算法
│ └── 专题五：排序（冒泡、选择、插入、sort）
│
└── 高级技巧
 └── 专题七：高精度加法
```

> **教师提示**：建议把这个框架图打印出来发给每位学生，或让学生自己在笔记本上画一遍。

专题一：函数（约 6 分钟）

知识要点速查表

知识点	说明	示例
函数定义	`返回类型 函数名(参数列表) { 函数体 }`	`int add(int a, int b) { return a + b; }`
函数调用	写函数名加小括号传入参数	`int s = add(3, 5);`
参数	函数的"输入口"，可以有多个	`void print(int x, int y)`
返回值	函数的"输出口"，用 `return` 返回	`return a + b;`
`void` 函数	不需要返回值的函数	`void sayHello() { cout << "Hi"; }`
函数声明	在 main 前面告知函数的存在	`int add(int a, int b);`

重要规则提醒

规则	正确	错误
有返回类型必须 return	`int f() { return 1; }`	`int f() { cout << 1; }`
调用时传参数个数要匹配	`add(3, 5)`	`add(3)`
函数要定义在调用之前（或先声明）	先定义/声明，后调用	先调用，后定义且无声明
形参是局部变量	函数内修改形参不影响外面	以为改了形参就改了原变量

快速回顾例子

```
#include <iostream>
using namespace std;

int maxVal(int a, int b) {
 if (a > b) return a;
 else return b;
}

int main() {
 int x = 7, y = 12;
 cout << maxVal(x, y) << endl; // 输出 12
 return 0;
}
```

> **提问互动**：如果我想写一个函数，传入三个数返回最大值，该怎么做？
> 答：可以在函数里调用两次 `maxVal`，比如 `return maxVal(maxVal(a, b), c);`，也可以用 if-else 直接比较三个数。

专题二：二维数组（约 5 分钟）

知识要点速查表

知识点	说明	示例
声明	`类型 数组名[行数][列数];`	`int a[3][4];`
下标	行和列都从 **0** 开始	`a[0][0]` 是第一行第一列
初始化	按行给初值	`int a[2][3] = { {1,2,3}, {4,5,6} };`
遍历	两层 for 循环：外层行，内层列	`for(i...) for(j...) a[i][j]`
应用场景	矩阵、棋盘、地图、表格	方阵转置、杨辉三角

核心理解

```
二维数组就像一个"表格":

 列0 列1 列2
行0 [ 1 ] [ 2 ] [ 3 ]
行1 [ 4 ] [ 5 ] [ 6 ]

a[1][2] = 6 第1行第2列
```

快速回顾例子

```
#include <iostream>
using namespace std;

int main() {
 int a[3][3] = { {1,2,3}, {4,5,6}, {7,8,9} };

 // 输出矩阵
 for (int i = 0; i < 3; i++) {
 for (int j = 0; j < 3; j++) {
 cout << a[i][j] << " ";
 }
 cout << endl;
 }

 // 求所有元素之和
 int sum = 0;
 for (int i = 0; i < 3; i++)
 for (int j = 0; j < 3; j++)
 sum += a[i][j];
 cout << "总和 = " << sum << endl; // 输出 45

 return 0;
}
```

> **提问互动**：一个 3×3 的矩阵，主对角线上的元素有什么特点？
> 答：主对角线上 `i == j`，即 `a[0][0]`、`a[1][1]`、`a[2][2]`。

专题三：枚举法（约 5 分钟）

知识要点速查表

知识点	说明
核心思想	**逐一尝试**所有可能的情况，检查哪些满足条件
适用场景	数据范围不大，可以"暴力"列举所有可能
关键步骤	① 确定枚举范围 ② 逐一检验 ③ 输出满足条件的
优化意识	缩小枚举范围、提前剪枝（跳过不可能的情况）

经典题型

题目	枚举思路
水仙花数	枚举 100~999，检查各位立方和
鸡兔同笼	枚举鸡的数量 0~总头数，算兔子
百钱百鸡	三层枚举公鸡、母鸡、小鸡数量
完美数	枚举因子，求因子之和

快速回顾例子：找水仙花数

```
#include <iostream>
using namespace std;

int main() {
 for (int n = 100; n <= 999; n++) {
 int ge = n % 10;
 int shi = n / 10 % 10;
 int bai = n / 100;
 if (ge*ge*ge + shi*shi*shi + bai*bai*bai == n) {
 cout << n << endl;
 }
 }
 return 0;
}
// 输出: 153 370 371 407
```

> **提问互动**：枚举法的优点和缺点分别是什么？
> 答：优点是思路简单不容易出错；缺点是数据太大时会很慢。

专题四：模拟法（约 5 分钟）

知识要点速查表

知识点	说明
核心思想	**按照题意一步一步做**，用代码"翻译"题目描述的过程
适用场景	题目描述了一个明确的操作流程
关键能力	① 读懂题意 ② 分解步骤 ③ 逐步翻译成代码
常见难点	边界处理、下标越界、循环条件

经典题型

题目	模拟思路
数字翻转	反复取末位、拼到新数上
约瑟夫问题	围成圈报数，标记出局
时钟问题	秒分时逐级进位
蛇形填数	按方向在矩阵中依次填数

快速回顾例子：数字翻转

```
#include <iostream>
using namespace std;

int main() {
 int n;
 cin >> n; // 例如输入 1234
 int rev = 0;
 while (n > 0) {
 rev = rev * 10 + n % 10; // 取末位拼到 rev 上
 n /= 10; // 去掉末位
 }
 cout << rev << endl; // 输出 4321
 return 0;
}
```

> **提问互动**：如果输入 1200，翻转后输出什么？
> 答：输出 `21`（前导零被自动省略，因为 `int` 类型不会保存前导零）。

专题五：排序（约 8 分钟）

三大手写排序对比

	冒泡排序	选择排序	插入排序
核心思想	相邻比较、逐个冒泡	每轮选最小的放前面	像打牌一样插入到正确位置
比喻	气泡往上浮	选最矮的排前面	打扑克牌整理手牌
外层循环	n-1 轮	n-1 轮	从第 2 个元素开始
内层操作	相邻两个比较交换	找到最小值的下标，交换	将当前元素插入到前面已排好的序列中
时间复杂度	$O(n^2)$	$O(n^2)$	$O(n^2)$

冒泡排序核心代码

```
for (int i = 0; i < n - 1; i++) { // n-1 轮
 for (int j = 0; j < n - 1 - i; j++) { // 每轮少比一个
 if (a[j] > a[j + 1]) {
 swap(a[j], a[j + 1]); // 相邻交换
 }
 }
}
```

选择排序核心代码

```
for (int i = 0; i < n - 1; i++) {
 int minIdx = i; // 假设当前位置最小
 for (int j = i + 1; j < n; j++) {
 if (a[j] < a[minIdx]) minIdx = j; // 找到更小的
 }
 swap(a[i], a[minIdx]); // 放到前面
}
```

插入排序核心代码

```
for (int i = 1; i < n; i++) {
 int key = a[i]; // 拿出当前牌
 int j = i - 1;
 while (j >= 0 && a[j] > key) { // 前面的牌比它大
 a[j + 1] = a[j]; // 往后挪
 j--;
 }
 a[j + 1] = key; // 插入到正确位置
}
```

sort 函数（快速武器！）

知识点	说明
头文件	`#include <algorithm>`
默认升序	`sort(a, a + n);`
降序排列	`sort(a, a + n, greater<int>());`
优点	一行搞定，速度快（O(n log n)）

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
 int a[5] = {5, 2, 8, 1, 4};
 sort(a, a + 5); // 升序: 1 2 4 5 8
 for (int i = 0; i < 5; i++) cout << a[i] << " ";
 return 0;
}
```

> **提问互动**：什么时候用手写排序？什么时候用 sort？
> 答：考试如果要求"写出排序过程"或"用冒泡排序"就必须手写；如果只要求"排序后输出结果"，用 sort 最方便快捷。

专题六：贪心算法（约 5 分钟）

知识要点速查表

知识点	说明
核心思想	**每一步都选择当前最优**（局部最优），希望最终得到全局最优
适用条件	问题具有"贪心选择性质"——局部最优能导致全局最优
一般步骤	① 排序（准备工作） ② 按某个策略逐个选择 ③ 得出结果
注意事项	不是所有问题都能用贪心，需要判断贪心策略是否正确

经典题型

题目	贪心策略
找零钱（最少硬币数）	每次选最大面值的硬币
活动安排（最多参加几个）	按结束时间排序，优先选结束早的
背包装物（部分可装）	按单位价值排序，优先装价值高的
排队打水（最小等待时间）	打水时间短的排前面

快速回顾例子：排队打水

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
 int n;
 cin >> n;
 int t[110];
 for (int i = 0; i < n; i++) cin >> t[i];

 sort(t, t + n); // 贪心：打水快的排前面

 long long total = 0;
 for (int i = 0; i < n; i++) {
 total += t[i] * (n - 1 - i); // 第 i 个人让后面 (n-1-i) 人等待
 }
 cout << total << endl;
 return 0;
}
```

> **提问互动**：为什么打水时间短的排前面，总等待时间最少？
> 答：因为排在前面的人会让后面所有人等待。把时间短的放前面，每个人让后面人多等的时间更少，所以总和最小。

专题七：高精度加法（约 6 分钟）

知识要点速查表

知识点	说明
为什么需要	`int` 最大约 21 亿，`long long` 最大约 9.2×10^{18}，再大就装不下了
核心思想	用**数组**模拟手算竖式加法：逐位相加、满十进一
存储方式	字符串读入 **倒序存入数组**（个位在 `a[0]`）
倒序原因	方便进位时在高位扩展

高精度加法步骤

```
步骤：
1. 用字符串读入两个大数
2. 倒序存入整型数组（个位在 a[0]）
3. 逐位相加 + 处理进位
4. 倒序输出结果
```

快速回顾例子

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
 string s1, s2;
 cin >> s1 >> s2;

 int a[210] = {0}, b[210] = {0}, c[210] = {0};
 int len1 = s1.length(), len2 = s2.length();

 // 倒序存入数组
 for (int i = 0; i < len1; i++) a[i] = s1[len1 - 1 - i] - '0';
 for (int i = 0; i < len2; i++) b[i] = s2[len2 - 1 - i] - '0';

 // 逐位相加
 int len = max(len1, len2);
 for (int i = 0; i < len; i++) {
 c[i] += a[i] + b[i];
 c[i + 1] += c[i] / 10; // 进位
 c[i] %= 10; // 保留个位
 }
 if (c[len] > 0) len++; // 最高位有进位

 // 倒序输出
 for (int i = len - 1; i >= 0; i--) cout << c[i];
 cout << endl;

 return 0;
}
```

关键细节提醒

```
字符转数字: '5' - '0' = 5（字符 '5' 的 ASCII 码减去 '0' 的 ASCII 码）
两个数位数可能不同，要取 max(len1, len2)
最后一位可能有进位，别忘了检查 c[len] > 0
```

> **提问互动**：为什么要倒序存储，而不是正序？
>
> 答：因为加法从个位开始算，如果正序存储，两个不同长度的数字"对不齐"（高位在前面对不上）。倒序存储后，`a[0]` 和 `b[0]` 都是个位，天然对齐！

五、答疑互动（5 分钟）

教师话术

> 七大专题都过了一遍！阶段二的内容比阶段一更有挑战性，大家有没有哪个知识点还不太清楚的？可以举手提问。

重点再强调（写在黑板角落）

```
 考试常见丢分点：
1. 函数返回类型和 return 语句不匹配
2. 二维数组下标别越界，行列都从 0 开始
3. 冒泡排序内层循环上界是 n-1-i，不是 n-1
4. sort 需要 #include <algorithm>
5. 高精度加法别忘了最高位进位的检查
6. 贪心题一定要先想清楚策略再写代码
```

六、综合测试（45 分钟）

> 发放测试卷（详见 `测试卷.md`），学生独立完成。
>
> **监考注意事项：**
> - 提醒学生先浏览全卷，合理分配时间
> - 读程写结果题建议用"模拟执行"的方法，一步一步跟踪变量变化
> - 编程题不要求运行，写在纸上即可，注意格式完整（头文件、main 函数）
> - 排序过程题需要写出每一步的数组状态

七、收卷 + 下节课预告（5 分钟）

教师话术

> 考卷大家交上来，老师批改之后下节课会讲评。
>
> 预告一下：下节课我们进入**阶段三**！第一个新知识就是**字符串**。
>
> 什么是字符串呢？其实我们已经见过了——就是 `"Hello World"` 这样一串字符组成的东西。
>
> 不过之前我们只是简单地用 `cout` 输出它，下节课我们要系统地学习：
> - 怎样用 **char 数组**存储字符串？
> - 怎样用 **string** 存储字符串？
> - 怎样获取字符串的长度、访问某个字符、拼接两个字符串？
>
> 学好字符串对信息学竞赛非常重要，很多题目都跟字符串处理有关。
>
> 下节课见！

八、教学反思（课后填写）

反思项目	记录
学生整体掌握情况	
哪些知识点遗忘率最高	
测试平均分 / 最高分 / 最低分	
函数和排序是否需要补充讲解	
高精度加法理解程度	
需要在下节课补充讲解的内容	
改进建议	